

MEDIATEK

MT8676 Android DebugLoggerUI

Version: 1.0
Release date: 2024-08-23

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-23	Harry Hu	Official release

Table of Contents

Version History	2
Table of Contents	3
List of Figures	3
List of Tables	3
1 DebugLoggerUI	4
1.1 Overview	4
1.1.1 Introduction	4
1.1.2 Abbreviation	4
1.2 Architecture/Process Overview	5
1.2.1 DebugLogger Introduction	5
1.2.2 Introduction to Rotation Mechanism of DebugLogger	5
1.2.3 Introduction to DebugLogger Storage	6
1.3 Configuration/Customization Guideline	6
1.3.1 Introduction to Control of DebugLogger	6
1.3.2 Introduction to DebugLogger Configuration	9
1.3.3 DebugLogger Settings Introduction	10
1.3.4 Trigger TagLog	12
1.3.5 DebugLogger Reference FAQ	13
1.4 Frequently Asked Questions/Troubleshooting	14
1.4.1 Debuglogger Precautions	14
Exhibit 1 Terms and Conditions	15

List of Figures

Figure 1-1. MTK log flow	5
--------------------------------	---

List of Tables

Table 1-1. Abbreviations	4
--------------------------------	---

1 DebugLoggerUI

1.1 Overview

1.1.1 Introduction

This chapter mainly introduces the architecture of DebugLogger, a log capturing tool for MediaTek, and some commonly used debugging methods when this log tool cannot work properly.

1.1.2 Abbreviation

Table 1-1. Abbreviations

Abbreviation	Explanation
boot__normal	Log before restart
bootprof	Time spent initializing critical processes
bsp_log	Process call log
connsyslog	BT/WiFi/GPS log
crash_log	Backtrace and process information when APPs crash
DebugLogger/MTKlog	A debugging tool designed by MediaTek to capture logs
DebugLoggerUI	Java layer recorder UI, which can start/stop logging tools and change some settings
events_log	Process creation/destruction logs, activity lifecycle logs
kernel_log	Kernel/driver log
Last_1_boot_normal	During the previous boot process, the boot log was not yet copied to the SD card or eMMC, and a reboot occurred. The previous boot log was saved in this folder, and the number 1 represents the number of boot attempts.
last_AndroidLog	Last Android log during shutdown
last_kmsg	Last Kernel log during shutdown
main_log	Native/java layer log
mblog_history	Mobile_log_d runing log
Mobile_log_d	Run the service at the bottom layer, mainly capturing records such as main/Kernel logs
Mobile log	Common log records
Modem log	Record the log of the modem
netlog	TCP dump log
pl_lk	Log of preloader/LK during startup phase
properties	All the properties of the device are usually queried in this file, including the phone version number and phone model information
radio_log	Log of communication system
scp_log	Log of SCP
sys_log	System log, e.g., AMS/WMS and other logs

1.2 Architecture/Process Overview

1.2.1 DebugLogger Introduction

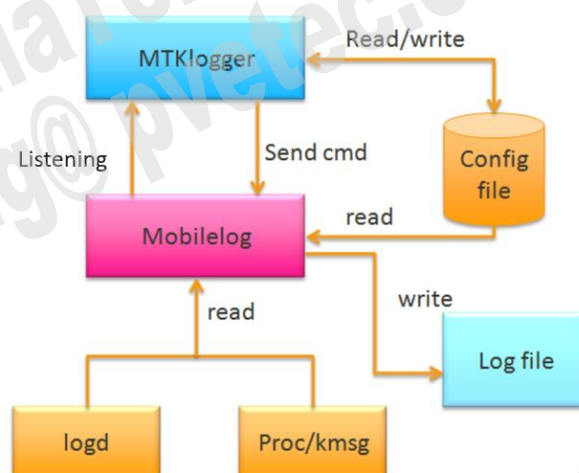


Figure 1-1. MTK log flow

The mechanism of the entire MTK log is composed of an upper layer of MTKlogger and a lower layer of mobile_log_d. The upper layer is responsible for issuing instructions, and the read/write config file calls the lower layer to run the program to achieve the expected purpose. The lower layer is the actual executor. After the program starts running, it will read the config file and execute commands based on the config file. mobile_log_d mainly reads the four log files of Android logd, as well as reads the kmsg/atvlog/scp logs under various proc nodes. The operations in this part of MTK log are actually the retrieval and integration of the existing logs, for example, ATF already has ATF log nodes, and logd itself is an Android log mechanism. After reading the log, a log file will be created to save various logs.

The general method of creating a log is to create a single ALog folder to save each type of log, which is read and written at a fixed rate in the form of a linked list. When the sum of all logs reaches the size limit of a single ALog folder, the logs inside will be rotated, and the nodes of the oldest log file will be deleted in the form of a linked list. The nodes of the linked list do not distinguish the types of logs which are rotated based on the ALog folder as the most basic unit. When the platform restarts or the upper level MTKlogger issues the start/stop command, a new ALog folder will be created. When the total size of ALog folders reaches a certain limit, the ALog folder of the oldest log file will be deleted. All content in the log file system is treated as files and can be accessed and manipulated through the file system.

1.2.2 Introduction to Rotation Mechanism of DebugLogger

The Mobile log in DebugLogger contains many ALogs. When stopping recording log and then starting recording log, a new ALog will be generated. When the platform restarts, the log will also be recorded in the new ALog after the restart.

Therefore, Mobile log has been set with two sizes:

One is the upper limit of the size of a single ALog folder. When the upper limit is reached, the previous log will be deleted (the boot normal log for a single boot will not be deleted), which is done as a rotate. Rotation is in chronological order, regardless of the type of log.

The other size is decided by an algorithm based on log_size (usually around a few megabytes), and all logs will be created as soon as they reach the upper limit. When the ALog reaches its maximum limit, the oldest log file created under this folder will be deleted.

1.2.3 Introduction to DebugLogger Storage

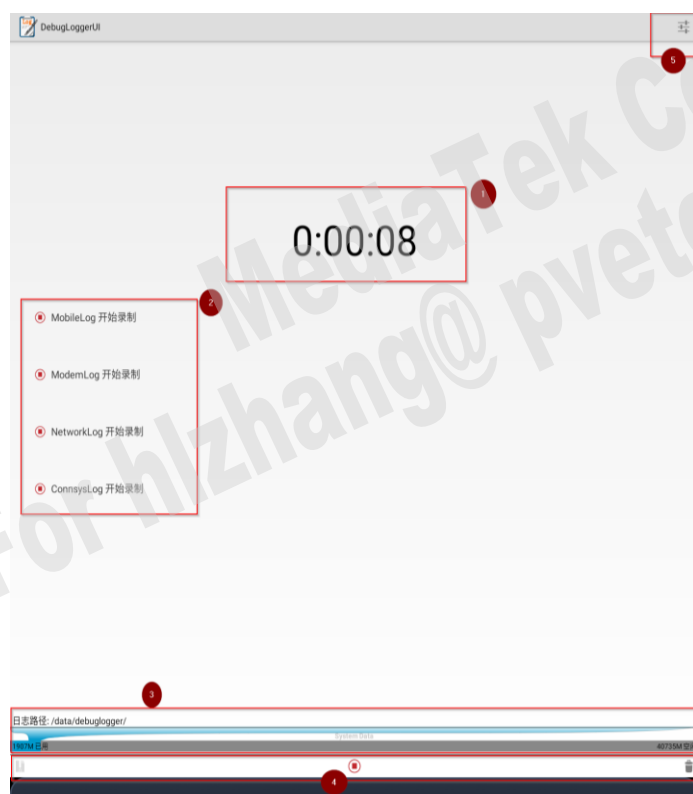
MTK log storage consists of three parts: first, logs are captured from various log buffers and stored in the MTK log buffer; second, logs from the MTK log buffer are saved to eMMC, and the file format has the suffix .curf; finally, the complete log is written to eMMC and the file format now does not has the .curf suffix.

Control the stored code (adjustment depends on the number of customer project logs):

```
logging.h :  
#define BUFFER_SIZE (256 * 1024) // 256k
```

1.3 Configuration/Customization Guideline

1.3.1 Introduction to Control of DebugLogger

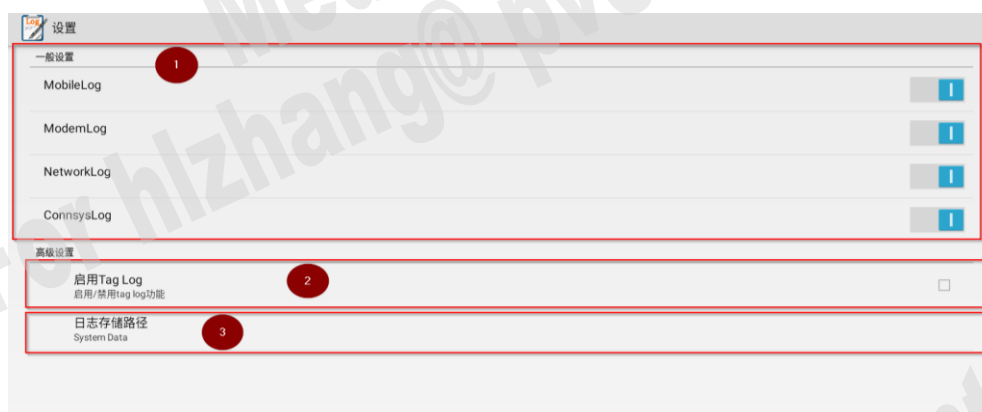


1.3.1.1 Main UI

1. Log running time: The recording time while capturing logs after recording starts.
2. Running status of each log: If a log is running, the left icon changes from a triangle to a square as shown in the figure above.

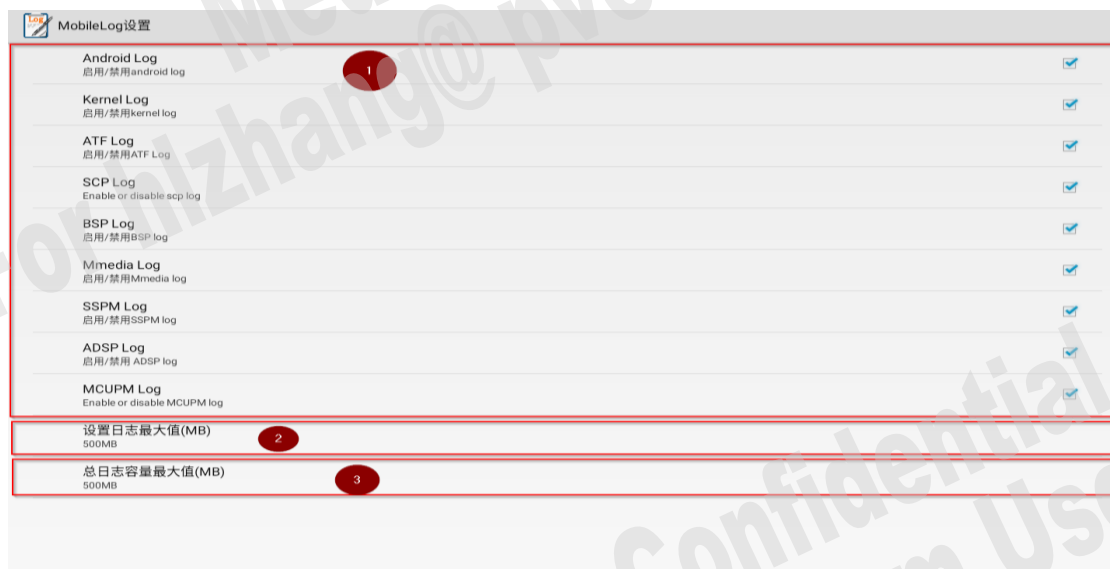
3. Log storage: The path where logs are stored and the remaining storage space for logs.
4. Control panel: The left button is used to start Tag Log. The trash can icon on the right is for clearing recorded logs, and the middle button is for starting or pausing recording.
5. Settings page entry: Click on this icon to access the detailed settings page, as shown in the figure below.

1.3.1.2 Log Setting



1. Customize the capture of different types of logs through the switch.
2. Enable or disable the Tag Log (When a system exception occurs and the DB is generated, Tag Log will save the platform logs at the time of the exception).
3. Log storage path

1.3.1.3 Mobile Log Setting



1. When Mobile log is enabled, different types of logs can be customized for capture by checking the corresponding options. If checked, the selected log types will be captured; if unchecked, the log will not be captured.
2. Set the maximum log buffer size for Mobile log. If the captured logs exceed this value, the previous logs will be deleted to free up space for the log buffer to be reused.

- Total log buffer size, including the currently recorded logs and all previous Mobile logs, that is, the total size of the folder where Mobile logs are stored.

1.3.1.4 Modem Log Setting

- Three modes for capturing Modem logs:
 - USB mode: Modem logs are captured by connecting via USB and using the ELT tool
 - SD mode: Modem logs are directly saved in the platform's `data/debuglogger` directory
 - Passive log to SD mode: Logs are captured for Modem exceptions and saved in the platform's `data/debuglogger` directory

- Automatically capture the logs when Modem exception occurs and save them in the platform's `data/debuglogger` directory
- Save location information in Modem1 logs
- Reset Modem automatically after EE occurs
- Total Modem log storage size

1.3.1.5 Network Log Setting

- Whether to check current Network connection status when Network log stops. If enabled, this operation will ping two IPs to confirm current connection status. These ping data may be very useful for analyzing network related issues, but it will take approximately 12 seconds to stop the log.

2. Set each Network log Package limit size
3. Total Network log folder size

1.3.2 Introduction to DebugLogger Configuration

Configuration file path: Before compilation, configure the default values of DebugLoggerUI through `mtklog configure bsp eng. prop&mtklog configure bsp user. prop file`. File path: `\device\media\common\mtklog`, `mtklog configure bsp eng. prop` controls the Eng version, while `mtklog configure bsp user. prop` controls the user and user debug versions.

1.3.2.1 Set the Default Path of Logs

Configure contents:

```
mtklog_path = system_data/device_storage/portable_storage
```

1.3.2.2 Set the Auto-start Log Types during the 1st Boot

```
com.mediatek.log.mobile.enabled = true
com.mediatek.log.modem.enabled = true
com.mediatek.log.net.enabled = true
com.mediatek.log.connsysfw.enabled = true
com.mediatek.log.gps host.enabled = true
com.mediatek.log.bthost.enabled = true
```

The eng version starts by default on the first boot, while other versions do not start by default and can be configured independently.

1.3.2.3 Set Log Rotate Size

```
com.mediatek.log.mobile.maxsize = 500 //Single mobile log folder size
com.mediatek.log.mobile.totalmaxsize = 1000 //Total mobile log size
com.mediatek.log.modem.maxsize = 2000
com.mediatek.log.net.maxsize = 600
com.mediatek.log.connsysfw.maxsize = 2000
com.mediatek.log.bthost.maxsize = 2000
```

You can modify the corresponding options as needed. Please note that the `com.mediatek.log.mobile.maxsize` should be smaller than `com.mediatek.log.mobile.total.maxsize`. The configured log cache size will be stored in a loop, that is, when the storage reaches its maximum value, if there are new logs, the previous old logs will be overwritten.

1.3.2.4 Set Sub Logs

```
com.mediatek.log.mobile.AllMode = true
```

Set all mobile sub logs if the specific type mobile sub log is not set.

```
com.mediatek.log.mobile.AndroidLog = true
com.mediatek.log.mobile.KernelLog = true
com.mediatek.log.mobile.SCPLog = true
com.mediatek.log.mobile.ATFLog = true
com.mediatek.log.mobile.BSPLog = true
com.mediatek.log.mobile.MmediaLog = true
com.mediatek.log.mobile.SSPMLog = true
com.mediatek.log.mobile.ADSPLog = true
```

If the mobile log sub item is not set, the default setting is `com. media. log. mobile. AllMode`.

1.3.2.5 Enable TagLog

```
com.mediatek.log.taglog.enabled = true
```

Default value: The eng version is true by default, while others are false.

1.3.2.6 Modem Log Auto-reset or Not When EE Occurs

```
com.mediatek.log.modem.autoreset.enabled = false
```

The default value is false.

A warning dialog will be shown if it is false.

1.3.2.7 Set the Startup Mode of Modem Log

```
com.mediatek.log.modem.mode = 2
```

1: USB mode, 2: sdcard mode, 3: PLS mode (Passive log to SD)

The default value is 2.

1.3.3 DebugLogger Settings Introduction

In addition to the home UI method, you can also directly use the adb method to control the debuglogger, which is suitable for customers who do not display the debuglogger interface.

1.3.3.1 Set the Start and Stop

Command format:

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name value --ei cmd_target
logTypes -n com.debug.loggerui/.framework.LogReceiver
```

value = start/stop

logTypes = 1 + 2 + 4 + 16 + 32 + 64

(MobileLog: 1, ModemLog: 2, NetworkLog: 4, GPSLog: 16, ConnsysFWLog: 32, BTHostLog: 64)

Example 1: Enable all logs

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name start --ei cmd_target 119
-n com.debug.loggerui/.framework.LogReceiver
```

Example 2: Close all logs

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name stop --ei cmd_target 119
-n com.debug.loggerui/.framework.LogReceiver
```

Note: The start and stop commands require a 15 second interval.

Example 3: Stop ModemLog & NetworkLog

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name stop --ei cmd_target 6 -n
com.debug.loggerui/.framework.LogReceiver
```

1.3.3.2 Query the Running Status

Command format:

```
adb shell getprop logstatusproperty
```

log status result: running with "1", stop with "0"

The logstatusproperty can have the following values:

logstatusproperty of mobile: vendor.MB.running

logstatusproperty of modem log: vendor.mdlogger.Running

logstatusproperty of network log: vendor.mtklog.netlog.Running

logstatusproperty of GPSLog: vendor.gpsdbglog.enable

logstatusproperty of BTHost log: vendor.bthcnoop.running

logstatusproperty of ConnsysFW log: vendor.connsysfw.running

eg: Obtain the status of the mobile log: adb shell getprop vendor.MB.running

1.3.3.3 Restart

Command format:

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name restart --ei cmd_target
logTypes -n com.debug.loggerui/.framework.LogReceiver
```

logTypes=1+2+4+16+32+64 (MobileLog: 1, ModemLog: 2, NetworkLog: 4, GPSLog: 16, ConnsysFWLog:32, BTHostLog: 64)

logTypes = -1 (all logs)

For example, only to start mobile log

Command:

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name restart --ei cmd_target 1
-n com.debug.loggerui/.framework.LogReceiver
```

Result: Only Mobile log restarts

1.3.3.4 Display the Modem EE Memory Dump Completion or Not

Command:

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name modem_auto_reset_value --ei cmd_target 2 -n com.debug.loggerui/.framework.LogReceiver
```

value = 1/0 (Enable/Disabled)

e.g., Display modem EE memory dump done dialog:

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name modem_auto_reset_1 --ei cmd_target 2 -n com.debug.loggerui/.framework.LogReceiver
```

1.3.3.5 Set Modem Log Mode

Command:

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name switch_modem_log_mode_value --ei cmd_target modemtype -n com.debug.loggerui/.framework.LogReceiver
```

value = 1/2/3 (USB/SD/PLS)

modemtype = 1/3 (md1/md3)

e.g., Set Modem1 Mode to SD

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name switch_modem_log_mode_2 --ei cmd_target 1 -n com.debug.loggerui/.framework.LogReceiver
```

1.3.3.6 Manually Trigger EE

Command:

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name force_modem_assert --ei cmd_target 2 -n com.debug.loggerui/.framework.LogReceiver
```

1.3.3.7 Enable/Disable TagLog

Command:

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name switch_taglog --ei cmd_target value -n com.debug.loggerui/.framework.LogReceiver
```

value = 1/0 (Enable/disable)

e.g., Enable TagLog

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name switch_taglog --ei cmd_target 1 -n com.debug.loggerui/.framework.LogReceiver
```

1.3.4 Trigger TagLog

Command:

```
adb shell am broadcast -a com.mediatek.log2server.EXCEPTION_HAPPEND -e path
SaveLogManually -e db_filename yourFileName --ez is_need_zip needZipValue --ez
is_need_all_logs needAllLogValue -n com.debug.loggerui/.framework.LogReceiver
```

Your file name can be defined by yourself.

needZipValue = true/false (zip file/not zip file)

needAllLogValue = true/false (tag all logs from MTK log folder/not tag all logs)

e.g., Tigger taglog to zip the current log

```
adb shell am broadcast -a com.mediatek.log2server.EXCEPTION_HAPPEND -e path SaveLogManually -e db_filename test
--ez is_need_zip true --ez is_need_all_logs false -n com.debug.loggerui/.framework.LogReceiver
```

1.3.4.1 Display DebugLoggerUI Main Interface

Start DebugLoggerUI Activity, command:

```
adb shell am start -n com.debug.loggerui/com.debug.loggerui.MainActivity
```

1.3.4.2 Set Log Size

1. Set Mobile log size

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name set_total_log_size_600 --ei
cmd_target 1 -n com.debug.loggerui/.framework.LogReceiver
```

2. Set Mobile log single APlog file size

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name set_log_size_300 --ei
cmd_target 1 -n com.debug.loggerui/.framework.LogReceiver
```

3. Set Modem log size

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name set_log_size_600 --ei
cmd_target 2 -n com.debug.loggerui/.framework.LogReceiver
```

4. Set Network log size

```
adb shell am broadcast -a com.debug.loggerui.ADB_CMD -e cmd_name set_log_size_200 --ei
cmd_target 4 -n com.debug.loggerui/.framework.LogReceiver
```

1.3.5 DebugLogger Reference FAQ

[FAQ13883] How to configure various parameters of MTKlogger

[FAQ12752] How to modify the default storage capacity size of MTKlogger

[FAQ19560] Special precautions for enabling MTKlog in the user version

[FAQ19362] How to set the size of the Mobile log, Modem log, Network log

[FAQ17814] How to set whether MTKlogger starts up automatically

Gain a deeper understanding of Logging Tools: <http://online.mediatek.com/QuickStart/QS00034>

1.4 Frequently Asked Questions/Troubleshooting

1.4.1 Debuglogger Precautions

1. It is not recommended for customers to customize MTK log in any way, e.g., modifying storage partitions or adding new features. You can submit questions to eservice for help from MediaTek.
2. It is not recommended for customers to perform any additional operations on MTK log when capturing logs, e.g., manually stopping MTK log service or modifying the MTK log storage path.
3. If the log is required, enter the command below

```
adb pull/data/debuglogger
```

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.